
Text Analysis I

Modern Approach: Feature Engineering

...

POLI3148 Data Science in Politics and Public Administration
Dr. Chen Haohan

Agenda



- Our November data science adventure
 - **Bonus Project Proposal:** due on 11/5
 - **Assignment 1:** due on 11/11
 - **In-class exercise 5:** to be assigned on 11/5, due on 11/18
 - **Assignment 2:** to be assigned on 11/19, due on 12/17
- Remaining lectures
 - Text analysis: *modern* approach
 - Text analysis: *modern* approach (con'd), *postmodern* approach
 - Machine learning: *modern* approach
 - Machine learning: *modern* approach (con'd), *postmodern* approach
 - *Final catch-up/ debriefing session*



Troubleshooting Tip with Colab Notebook

- Splitting code into different code chunk help you identify the problema
 - Make fragmented code chunks when you test your code
 - Then merge them for final outputs
- Libraries need to be loaded only ONCE at the start of the notebook



Objective of Text Analysis

We use machine to augment our capacity to **READ** text

What do we do when we read?

- Summarize the text
 - Look for specific contents of interest
 - Search for text related to the current one
-

Unit of Analysis in Text Analysis

At which level do you want learn ?

- Corpus (i.e., the entire collection of documents)
 - What contents/ concepts are in this corpus? How frequent does each one appear?
 - How frequently does the corpus discuss certain topic/ concept of your choice?
- Document
 - What contents/ concepts are in this document? How frequent does each one appear?
 - How frequently does the document discuss certain topic/ concept of your choice?
 - Which documents is most similar to the document of interest?
- Part of document (e.g., paragraph, sentence, word, phrase)
 - Which paragraphs/ sentences/ words/ phrases are similar to the one of interest?

Text Analysis I: Modern Approach

What do I call it a “*modern* approach?”

- Methods for text data before the recent boom of Large Language Models (e.g., ChatGPT) (aka, the “*postmodern* approach”)
- Essential for forming basic understanding of text analysis
- Will be useful for years even with new techniques
 - Computationally efficient
 - Good for data safety
 - Inexpensive

Map skills into our data science workflow

- **Data Input**
 - Load text data
- **Data processing**
 - Extract linguistic features from text
 - Extract substantive features from text
- **Data Output**
 - Save processed text data
 - Visualize results of text analysis



Map skills into our data science workflow

- **Data Input**
 - **Load text data**
- **Data processing**
 - Extract linguistic features from text
 - Extract substantive features from text
- **Data Output**
 - Save processed text data
 - Visualize results of text analysis



Load text data

- Text data can come in various formats
- The simplest format: Text in tables
 - Excel
 - CSV
- Other possible formats
 - TXT
 - JSON
 - XML
 - HTML
 - Databases: SQL etc.

We work with the **simplest** format in class. You may come across other formats elsewhere. Don't be intimidated. Remember there is nothing mysterious about these other formats. They were invented for efficient data storage and retrieval. Python has functions to work with essentially all popular formats. Just look it up.

Chinese Ministry of Foreign Affairs Press Conferences Corpus

- Data: <https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/BAKGET>
- Paper: <https://link.springer.com/article/10.1007/s11366-021-09762-3>

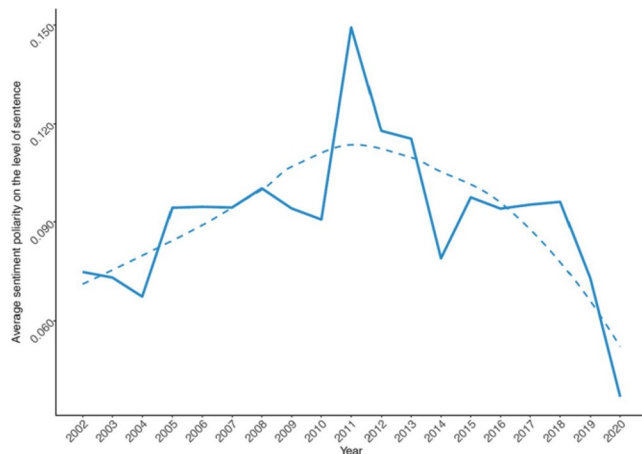


Fig. 4 Change in sentiment polarity towards the US in Chinese foreign policy discourse. Note: Solid line visualizes raw sentiment averages on the level of sentences. Dashed line visualizes smoothed trend (using loess function [Local Polynomial Regression Fitting])

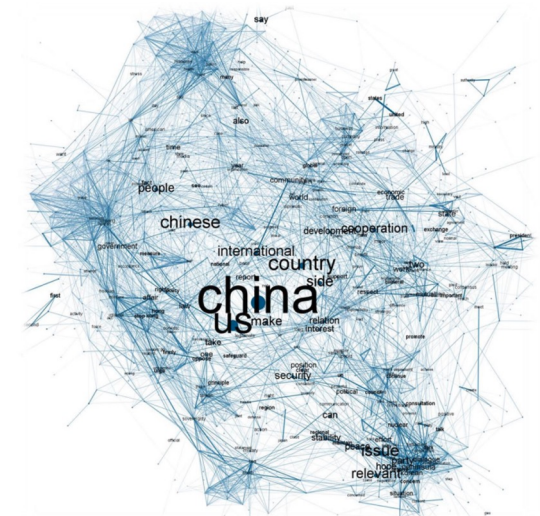


Fig. 6 Semantic network tracing US-China discourse under President Xi Jinping

Warm-up Exercise (10 minutes)

1. Set up a project folder in Google Drive (mind the structure)
2. [Download](#) the data from the website
3. Unzip the folder, upload all contents to your project folder on Google Drive
4. Start a Colab Notebook
5. Load the data into your notebook (use the Excel file)
6. Referring to the codebook, write Python code to generate the following summary statistics
 - a. How many rows and columns are in the dataset?
 - b. What are the names of the columns?
 - c. Who are the spokespersons?
 - d. What are the top 10 news agencies who asked the most questions?
 - e. What time period does the data cover?
 - f. *Ask a few questions regarding what you are curious to learn from this dataset*

Hints

- Function to load the dataset
`df = pd.read_excel("[YOUR PATH]/CMFA_PressCon_v4.xlsx")`
- Functions for descriptive statistics
`df.columns`
`df.xx.value_counts()`
`df.yy.describe()`
- Process the **date** variable
`df.loc[:, 'date_t'] = pd.to_datetime(df.date, errors = 'coerce').`
`df.date_t.describe()`

Map skills into our data science workflow

- **Data Input**
 - Load text data
- **Data processing**
 - Extract linguistic features from text
 - Extract substantive features from text
- **Data Output**
 - Save processed text data
 - Visualize results of text analysis



Learn from our case: What features have been extracted from the MoFA text?

Take a look at an example

`df.loc[0, :].to_dict()`

👀 Examining one or multiple record(s) of your dataset is always a good way to start any data analysis

Locations, persons, organizations, sentiment in the question and answer respectively

How can we extract these from the long piece of text?

Extract linguistic features from text

Extracting linguistic feature (“linguistic annotation”) is an essential pre-step in the modern text analysis approach. Techniques that we are primarily interested in include:

- **Tokenization:**
Segmenting text into words, punctuations marks etc.
- **Stemming/ Lemmatization:**
Assigning the base forms of words. For example, the lemma of “was” is “be”, and the lemma of “rats” is “rat”.
- **Part-of-speech tagging:**
Assigning word types to tokens, like verb or noun.
- **Stopword removal**
Remove words that occur commonly across documents. They are typically not substantively meaningful. Examples: “the” “a” “to”

Extract substantive features from text

Extract information that can answer your research questions of interest. What you would like to extract from text depends on your research question. In this application, we are first interested in the **Named Entities** mentioned.

Named Entity Recognition: Labelling named “real-world” objects, like persons, countries, companies, locations etc.

Python Library: spaCy

A tool for extracting linguistic and substantive features with only a few lines of code

<https://spacy.io/>

Very friendly for beginners

- Free
 - Easy to use
 - Multifunctional
 - Multilingual
-

Loading the library, then load a language model

```
import spacy
```

```
nlp = spacy.load("en_core_web_sm")
```

"en_core_web_sm"

- **en**: English
- **core**: a general-purpose language model
- **web**: the "pipeline" is trained on web data (as opposed to news data)
- **sm**: "small" size (as opposed to md, lg, trf)

Extract Linguistic Features from the first question in the dataset

```
df.loc[0, 'question']
```

```
doc = nlp(df.loc[0, 'question'])
```

The function separate your text into tokens, and get (primarily) linguistic features for each token. You can operate it as operating a *list*. Try:

```
doc[0]
```

```
doc[1]
```

```
for word in doc:
```

```
    print(word)
```

```
[word for word in doc]
```

This is a “Pythonic” way to write a **for loop**. Essentially, you ask Python to get each element of doc, then put them into a **list**.

Get Linguistic Features

- Get raw tokens
 `[word for word in doc]`
 or
 `[word.text for word in doc]`
- Get low-cased tokens
 `[word.lower for word in doc]`
- Get lemmatized tokens
 `[word.lemma for word in doc]`
- Get part-of-speech tags of tokens (What do the tags mean? [Read this](#))
 `[word.pos for word in doc]`
- Get stopwords indicators for tokens
 `[word.is_stop for word in doc]`

Make a table containing all linguistic features?

```
pd.DataFrame(  
    {  
        'token': [word for word in doc],  
        'lower': [word.lower_ for word in doc],  
        'lemma': [word.lemma_ for word in doc],  
        'pos': [word.pos_ for word in doc],  
        'stopword': [word.is_stop for word in doc]  
    }  
)
```

	token	lower	lemma	pos	stopword
0	How	how	how	SCONJ	True
1	many	many	many	ADJ	True
2	Hong	hong	Hong	PROPN	False
3	Kong	kong	Kong	PROPN	False
4	and	and	and	CCONJ	True
5	Taiwan	taiwan	Taiwan	PROPN	False
6	people	people	people	NOUN	False
7	were	were	be	AUX	True
8	injured	injured	injure	VERB	False
9	in	in	in	ADP	True
10	the	the	the	DET	True
11	blasts	blasts	blast	NOUN	False
12	in	in	in	ADP	True
13	the	the	the	DET	True
14	Bali	bali	Bali	PROPN	False
15	Island	island	Island	PROPN	False
16	,	,	,	PUNCT	False
17	Indonesia	indonesia	Indonesia	PROPN	False
18	?	?	?	PUNCT	False
19	Are	are	be	AUX	True
20	there	there	there	PRON	True

Extract Substantive Features (Named Entities) from Text

Note: Data are returned with a different logic. Linguistic features are returned at token-level. Substantive features are return in document-level

- Named Entities are in **doc.ents**
- Named Entities
[ne.text for ne in doc.ents]
- Lemmatized Named Entities
[ne.lemma_ for ne in doc.ents]
- Types of Named Entities
[ne.label_ for ne in doc.ents]
- Starting and ending positions (among tokens)
[ne.start for ne in doc.ents]
[ne.end for ne in doc.ents]
- Starting and ending positions (among characters)
[ne.start_char for ne in doc.ents]
[ne.end_char for ne in doc.ents]

Make a table containing all the Named Entities?

```
pd.DataFrame(  
    {  
        'ne': [ne.text for ne in doc.ents],  
        'ne_lemma': [ne.lemma_ for ne in doc.ents],  
        'ne_label': [ne.label_ for ne in doc.ents],  
        'ne_start': [ne.start for ne in doc.ents],  
        'ne_end': [ne.end for ne in doc.ents],  
        'ne_start_char': [ne.start_char for ne in doc.ents],  
        'ne_end_char': [ne.end_char for ne in doc.ents]  
    }  
)
```

	ne	ne_lemma	ne_label	ne_start	ne_end	ne_start_char	ne_end_char
0	Hong Kong	Hong Kong	GPE	2	4	9	18
1	Taiwan	Taiwan	GPE	5	6	23	29
2	the Bali Island	the Bali Island	LOC	13	16	67	82
3	Indonesia	Indonesia	GPE	17	18	84	93
4	Mainland	Mainland	NORP	22	23	109	117
5	the Chinese Embassy	the Chinese Embassy	ORG	28	31	145	164
6	Indonesia	Indonesia	GPE	32	33	168	177

Your Turn: Extract Linguistic and Substantive Features from the first *answer*

Extract the linguistic and substantive features from the answer to the question we just analyzed. Apply the same method we just discussed.

5 minutes



Extracting Features from a Collection of Documents

We have just performed all these analyses to one document.

What shall we do are to scale it up for a collection of documents?

For example: All the questions asked in the MoFA conferences in **2023**.



Steps

1. Create a subset of the dataframe covering Q&A in 2023
 2. Locate the “Question” column
 3. Repeatedly perform the same text analysis to every question
-

Data Preparation

Get the 2023 subset

```
df_2023 = df.loc[df.year == 2023, :]
```

Create containers to store the results

```
result_ling_ls = []
```

```
result_ner_ls = []
```

Now, a gigantic code chunk to batch-process text analysis

```
result_ling_ls = []
result_ner_ls = []

for q_id, q_text in zip(df_2023.id, df_2023.question):
    doc = nlp(q_text)

    r_ling = pd.DataFrame(
        {'token': [word for word in doc],
         'lower': [word.lower_ for word in doc],
         'lemma': [word.lemma_ for word in doc],
         'pos': [word.pos_ for word in doc],
         'stopword': [word.is_stop for word in doc]})
    r_ling['id'] = q_id
    r_ling['analysis_type'] = 'linguistic'

    r_ner = pd.DataFrame(
        {'ne': [ne.text for ne in doc.ents],
         'ne_lemma': [ne.lemma_ for ne in doc.ents],
         'ne_label': [ne.label_ for ne in doc.ents],
         'ne_start': [ne.start for ne in doc.ents],
         'ne_end': [ne.end for ne in doc.ents],
         'ne_start_char': [ne.start_char for ne in doc.ents],
         'ne_end_char': [ne.end_char for ne in doc.ents]})
    r_ner['id'] = q_id
    r_ner['analysis_type'] = "ner"

    result_ling_ls.append(r_ling)
    result_ner_ls.append(r_ner)
    print(q_id)
```

Can you make a *function* to make the code more readable?

Turn Outputs into Tables

Turn outputs into a format we are familiar with: a Pandas dataframe

```
result_ling_df = pd.concat(result_ling_ls, ignore_index = True)
```

```
result_ner_df = pd.concat(result_ner_ls, ignore_index = True)
```

	token	lower	lemma	pos	stopword	id	analysis_type
0	opening_remark	opening_remark	opening_remark	X	False	28448	linguistic
1	The	the	the	DET	True	28449	linguistic
2	2023	2023	2023	NUM	False	28449	linguistic
3	New	new	New	PROPN	False	28449	linguistic
4	Year	year	Year	PROPN	False	28449	linguistic
...	...	...	...	...	...	...	...
124522	do	do	do	VERB	True	30964	linguistic
124523	with	with	with	ADP	True	30964	linguistic
124524	those	those	those	DET	True	30964	linguistic
124525	people	people	people	NOUN	False	30964	linguistic
124526	?	?	?	PUNCT	False	30964	linguistic

124527 rows x 7 columns

	ne	ne_lemma	ne_label	ne_start	ne_end	ne_start_char	ne_end_char	id	analysis_type
0	2023	2023	DATE	1.0	2.0	4.0	8.0	28449	ner
1	New Year Address	New Year Address	EVENT	2.0	5.0	9.0	25.0	28449	ner
2	Xi Jinping	Xi Jinping	PERSON	7.0	9.0	39.0	49.0	28449	ner
3	China	China	GPE	39.0	40.0	226.0	231.0	28449	ner
4	China	China	GPE	62.0	63.0	363.0	368.0	28449	ner
...	...	...	...	...	...	...	...	...	...
15646	Chinese	chinese	NORP	40.0	41.0	218.0	225.0	30963	ner
15647	Laukkaing	laukkae	GPE	43.0	44.0	239.0	248.0	30963	ner
15648	China	China	GPE	49.0	50.0	271.0	276.0	30963	ner
15649	Laukkaing	Laukkaing	GPE	8.0	9.0	40.0	49.0	30964	ner
15650	China	China	GPE	16.0	17.0	88.0	93.0	30964	ner

15651 rows x 9 columns

IMPORTANT: What have we achieved?

We have just turned *unstructured* data into *structured* data

Understand the Documents Using the Created Features

Gain insights into the whole corpus

- Most frequent tokens
 - Using all tokens
 - Subset non-stopword
 - Remove other not useful words with part-of-speech tagging
- Most frequent Named Entities
 - Overall
 - Within categories

Understand the Documents Using the Created Features

Merging with the documents' *metadata* to gain further insights

```
result_ling_df_merge = pd.merge(result_ling_df, df, on = "id", how = "inner")
```

Do you get why we keep the "id"
column

Important: Always keep your
processed text with unique identifiers.

Ask more questions with the merged dataset

How did the contents of questions vary ...

- Across time?
 - Across news media?
 - ...
-

**Your Turn: Perform
analysis on answers in
2023**

Map skills into our data science workflow

- **Data Input**
 - Load text data
- **Data processing**
 - Extract linguistic features from text
 - Extract substantive features from text
- **Data Output**
 - **Save processed text data**
 - Visualize results of text analysis



Save processed text

Save the the structured text data, so that you will not have to re-do the feature extraction everytime.

Use pandas functionalities:

```
result_ling_df.to_csv(os.path.join(PATH_PROJECT, "data_processed/text_question_ling.csv"), index = False)
```

```
result_ne_df.to_csv(os.path.join(PATH_PROJECT, "data_processed/text_question_ne.csv"), index = False)
```

Why save processed text?

Generating text features is just the first step.
We re-use them (for many times) for various downstream analyses.
Saving these features help with efficient data processing.



Aside



TRAINED PIPELINES

Catalan
Chinese
Croatian
Danish
Dutch
English
Finnish
French
German
Greek
Italian
Japanese
Korean
Lithuanian
Macedonian
Multi-language
Norwegian Bokmål
Polish
Portuguese
Romanian
Russian
Slovenian
Spanish
Swedish
Ukrainian

Try it yourself: Apply it to a different language?

```
spacy.cli.download("zh_core_web_sm")
```

```
nlp = spacy.load("zh_core_web_sm")
```

Caveat: Linguistic features that we can generate for English need not be available for other language. For example, there is no lemmatization for Chinese.

Read the documentation. But the documentation is not always informative. Poke around. Try.